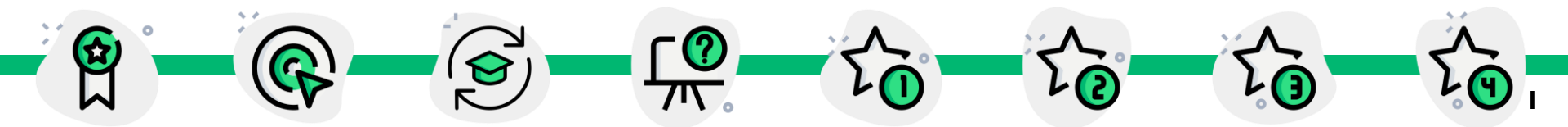


「2026년도 1학기 한국성서대학교 AI융합학부」

알고리즘

2026.04.13

6주차



재귀 알고리즘

05-1 재귀의 기본

05-2 재귀 알고리즘의 분석

05-3 하노이의 탑

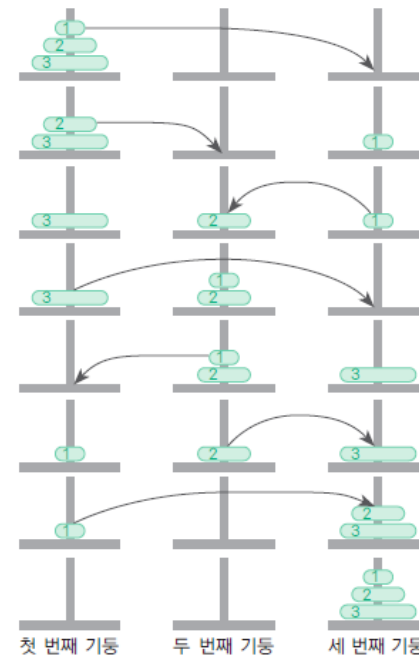


05-3 하노이의 탑

◆하노이의 탑 살펴보기 - (1)

• 하노이의 탑 Towers of Hanoi

- 작은 원반이 위에, 큰 원반이 아래에 위치할 수 있도록 원반을 3개의 기둥 사이에서 옮기는 문제
- 모든 원반은 크기가 다르고 처음에는 모든 원반이 이 규칙에 맞게 첫 번째 기둥에 쌓여 있음
- 이 상태에서 모든 원반을 세 번째 기둥으로 최소의 횟수로 옮기면 됨
- 이때 원반은 1개씩만 옮길 수 있고, 큰 원반을 작은 원반 위에 쌓을 수는 없음



원반 1을 첫 번째 기둥에서 세 번째 기둥으로

원반 2를 첫 번째 기둥에서 두 번째 기둥으로

원반 1을 세 번째 기둥에서 두 번째 기둥으로

원반 3을 첫 번째 기둥에서 세 번째 기둥으로

원반 1을 두 번째 기둥에서 첫 번째 기둥으로

원반 2를 두 번째 기둥에서 세 번째 기둥으로

원반 1을 첫 번째 기둥에서 세 번째 기둥으로

[그림 5-8] 하노이의 탑(원반 3개)



05-3 하노이의 탑

◆ 하노이의 탑 살펴보기 - (2)

• 원반을 옮기는 순서

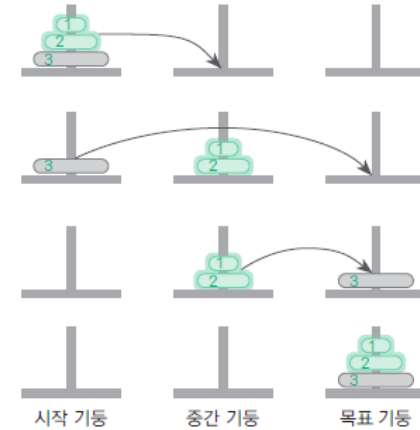
- 처음에 원반이 놓인 기둥을 '시작 기둥', 목적지의 기둥을 '목표 기둥', 남은 중간의 기둥을 '중간 기둥'이라 함
- 원반 1과 원반 2가 겹친 것을 '그룹'이라 함
- 가장 큰 원반을 최소의 단계로 목표 기둥으로 옮기려면 가장 먼저 '그룹'을 중간 기둥으로 옮겨야 하는데 이는 총 3단계로 완료됨
- 원반 1과 원반 2가 겹친 '그룹'을 옮기기 위해 원반 1만 '그룹'으로 보면 이전과 같이 3단계로 구현할 수 있음

1 그룹을 시작 기둥에서 중간 기둥으로

2 원반 3을 시작 기둥에서 목표 기둥으로

3 그룹을 중간 기둥에서 목표 기둥으로

3단계로 완료



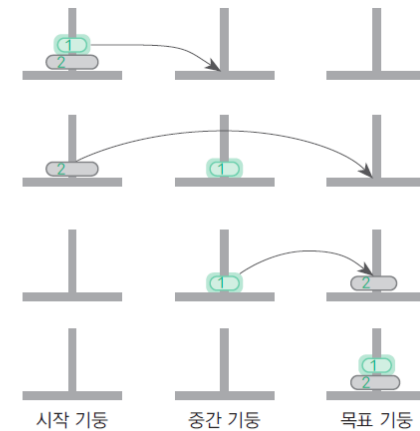
[그림 5-9] 하노이의 탑 풀이(원반 3개)

1 그룹을 시작 기둥에서 중간 기둥으로

2 원반 3을 시작 기둥에서 목표 기둥으로

3 그룹을 중간 기둥에서 목표 기둥으로

3단계로 완료



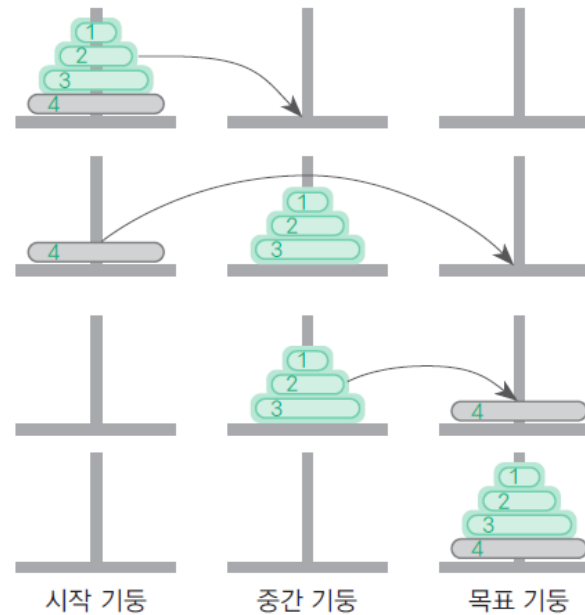
[그림 5-10] 하노이의 탑 풀이(원반 2개)



05-3 하노이의 탑

◆하노이의 탑 살펴보기 - (3)

- 원반이 4개일 경우
 - 원반 1, 원반 2, 원반 3의 총 3개를 겹친 상태를 '그룹'으로 보면 다시 같은 방법(3단계)으로 옮길 수 있음
 - 그리고 다시 3개의 '그룹'을 옮기면 되는데, 이 경우는 이미 앞에서 해결함
 - 이 방법으로 원반이 N개인 하노이의 탑 문제를 해결할 수 있음



1 그룹을 시작 기둥에서 중간 기둥으로

2 원반 4를 시작 기둥에서 목표 기둥으로

3 그룹을 중간 기둥에서 목표 기둥으로

3단계로 완료

[그림 5-11] 하노이의 탑 풀이(원반 4개)



05-3 하노이의 탑

◆ 하노이의 탑 살펴보기 - (4)

- 실습 5-7
 - 하노이의 탑을 구현하는 프로그램
 - move 함수의 매개변수 no는 옮겨야 할 원반의 개수, x는 시작 기둥의 번호, y는 목표 기둥의 번호
 - 기둥 번호는 정수 1, 2, 3으로 나타냄
 - 기둥 번호의 합이 6이므로 시작 기둥, 목표 기둥이 어느 기둥이더라도 중간 기둥은 $6 - x - y$ 로 구할 수 있음
 - 바닥 원반을 제외한 그룹(원반[1] ~ 원반[no - 1])을 시작 기둥에서 중간 기둥으로 옮김
 - 바닥 원반 no를 시작 기둥에서 목표 기둥으로 옮겼음을 출력
 - 바닥 원반을 제외한 그룹(원반[1] ~ 원반[no - 1])을 중간 기둥에서 목표 기둥으로 옮김



05-3 하노이의 탑

◆ 실습 5-7 – 하노이의 탑

Chapt05_실습 5-7/hanoi.c

```
#include <stdio.h>
```

```
/*--- 원반[1] ~ 원반[no]를 x 기둥에서 y 기둥으로 옮김 ---*/
```

```
void move(int no, int x, int y)
```

```
{  
    if (no > 1)  
        move(no - 1, x, 6 - x - y); // 그룹을 시작
```

```
기둥에서 중간 기둥으로
```

```
    // 바닥 원반을 목표 기둥으로
```

```
    printf("원반[%d]를(을) %d 기둥에서 %d  
기둥으로 옮김\n", no, x, y);
```

```
    if (no > 1)  
        move(no - 1, 6 - x - y, y); // 그룹을 중간  
기둥에서 목표 기둥으로
```

```
}
```

```
int main(void)
```

```
{
```

```
    int n;    // 원반의 개수
```

```
    printf("하노이의 탑\n원반 개수: ");
```

```
    scanf("%d", &n);
```

```
    move(n, 1, 3); //원반의 개수, 출발 기둥 번호,  
도착 기둥 번호
```

```
    return 0;
```

```
}
```



05-3 하노이의 탑

◆하노이의 탑 살펴보기 - (5)

- no가 3일 때 move 함수의 동작



[그림 5-12] move 함수의 실행 과정($n = 3$ 일 경우)

정렬

◆ 목차

06-1 정렬

06-2 버블 정렬

06-3 단순 선택 정렬

06-4 단순 삽입 정렬



06-1 정렬

◆ 정렬 sorting

- 이름, 학번, 키 등 핵심 항목(key)의 대소 관계에 따라 데이터 집합을 일정한 순서로 줄지어 늘어서도록 바꾸는 작업
- 키값이 작은 데이터를 앞쪽에 놓으면 오름차순(ascending order) 정렬, 그 반대로 놓으면 내림차순(descending order) 정렬이라고 부름

◆ 정렬 알고리즘의 안정성

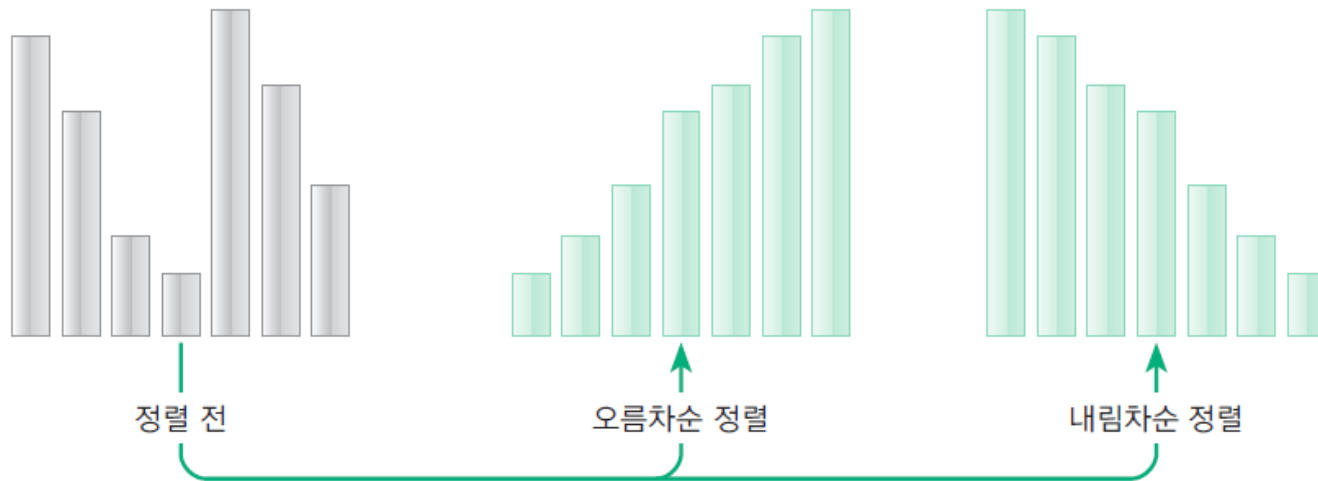
- 안정된 정렬이란 같은 값의 키를 가진 요소의 순서가 정렬 전후에도 유지되는 것

◆ 정렬 알고리즘의 핵심 요소

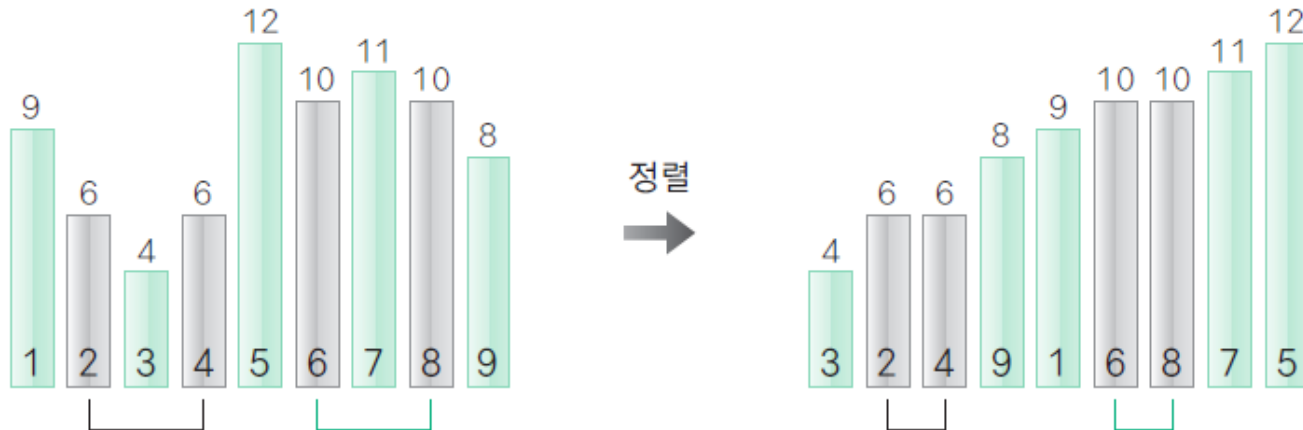
- 교환, 선택, 삽입



06-1 정렬



[그림 6-1] 오름차순 정렬과 내림차순 정렬



같은 키값인 요소의 순서가 정렬 전후에도 유지됩니다.

[그림 6-2] 안정된 정렬



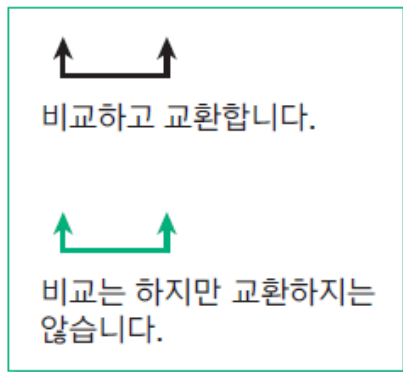
06-2 버블 정렬

◆ 패스_{pass}

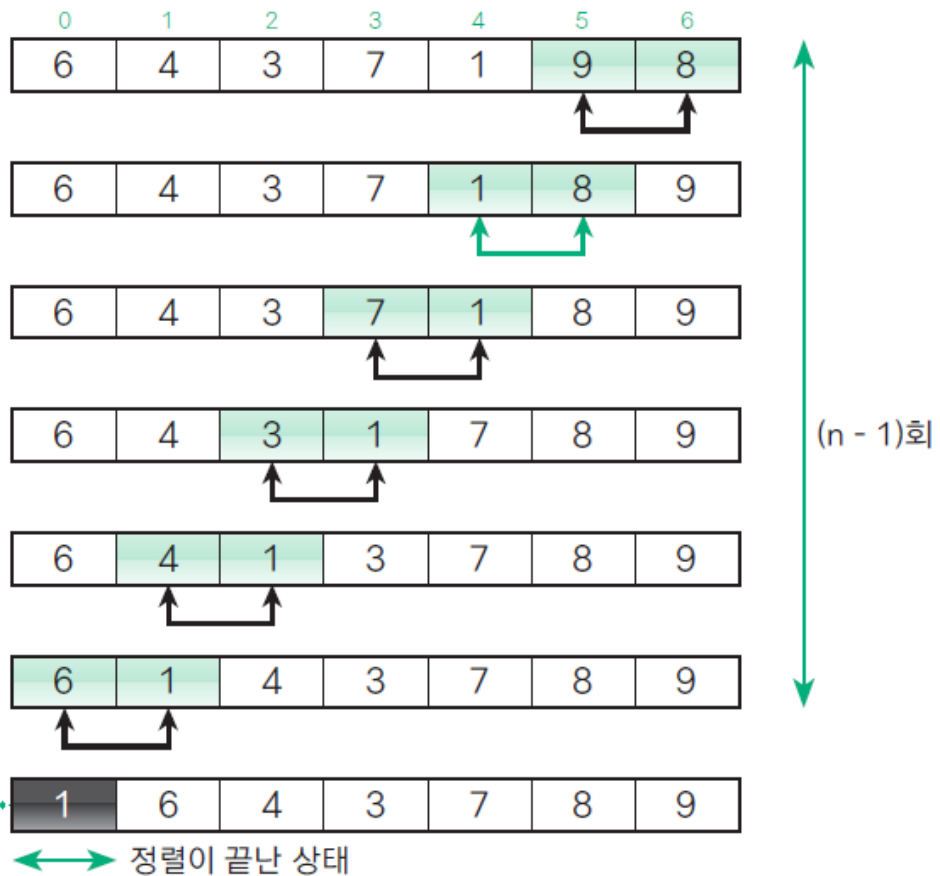
- 이웃한 요소를 비교하고 교환하는 작업을 첫 번째 요소까지 계속 하면 그림 6-3과 같은 상태가 됨
- 요소의 개수가 n 개인 배열에서 $n-1$ 회 비교, 교환을 하고 나면 가장 작은 요소가 맨 처음으로 이동함
- 이런 일련의 과정(비교, 교환 작업)을 패스(pass)라고 함



06-2 버블 정렬



가장 작은 요소가 맨 처음으로 이동합니다.



[그림 6-3] 버블 정렬의 첫 번째 패스



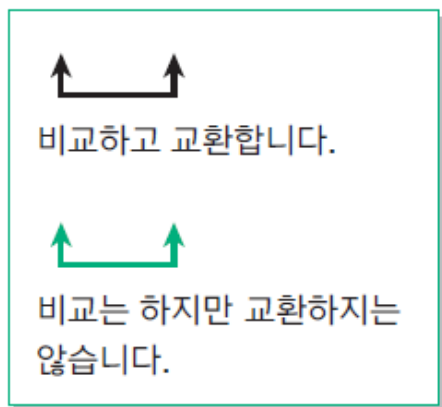
06-2 버블 정렬

◆ 패스 수행 횟수

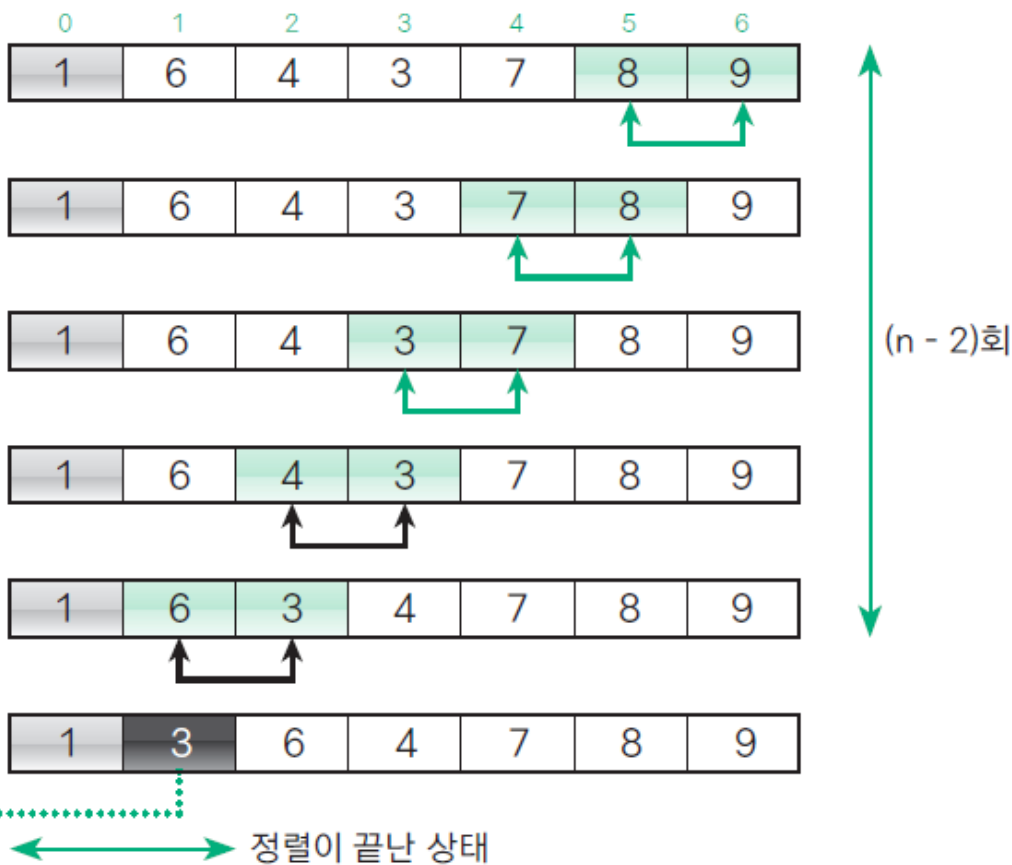
- 두 번째 패스의 비교 횟수는 첫 번째 패스보다 1회 적은 $n - 2$ 회
- 왜냐하면 패스를 1회 수행할 때마다 정렬할 요소가 하나씩 줄어들기 때문
- 패스를 k 회 수행하면 앞쪽의 요소 k 개가 정렬
- 모든 정렬이 끝나려면 $n - 1$ 회의 패스가 수행되어야 함



06-2 버블 정렬



3이 2번째 위치로 이동합니다.



[그림 6-4] 버블 정렬의 두 번째 패스



06-2 버블 정렬

◆ 버블 정렬 알고리즘

- 비교하는 두 요소의 인덱스를 $j - 1$ 과 j 라 함
- 배열의 끝(오른쪽)부터 스캔하기 때문에 j 의 시작값은 $n - 1$
- 한 번의 패스에서는 j 값이 $i + 1$ 이 될 때까지 비교, 교환을 수행하면 됨
- 비교 횟수는 첫 번째 패스는 $n - 1$ 회, 두 번째 패스는 $n - 2$ 회, ... 이므로 $n(n - 1) / 2$ 회
- 실제 요소를 교환하는 횟수는 배열의 요소값에 더 많이 영향을 받기 때문에 교환 횟수의 평균값은 비교 횟수의 절반인 $n(n - 1) / 4$ 회
- 또한 swap 함수 안에서 값의 이동이 3회 발생하므로 이동 횟수의 평균은 $3n(n - 1) / 4$ 회

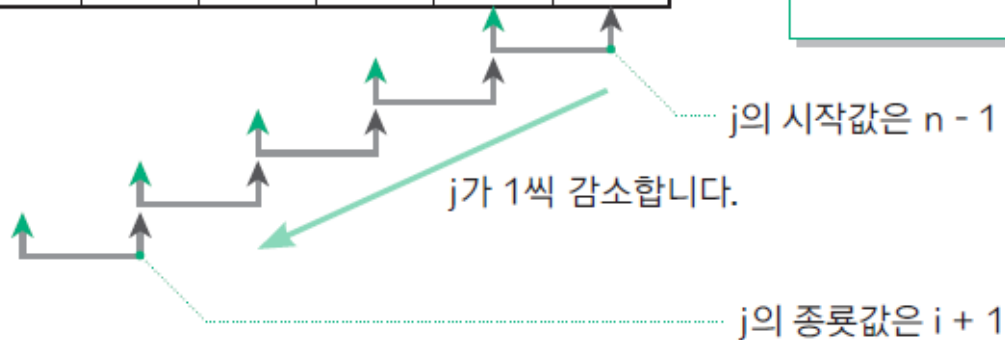


06-2 버블 정렬



$a[j-1]$ 과 $a[j]$ 를 비교합니다.

앞쪽 i 개의 요소
 $a[0] \sim a[i-1]$ 은
정렬이 끝난 상태



[그림 6-5] 버블 정렬의 i 번째 패스



06-2 버블 정렬

◆ 실습 6-1 - 버블정렬 (버전1)

Chapt06_실습 6-1/bubble1.c

// 버블 정렬(버전 1)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define swap(type, x, y) do { type t = x; x = y; y = t; }  
while (0)
```

/*--- 버블 정렬 ---*/

```
void bubble(int a[], int n)
```

```
{  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = n - 1; j > i; j--)  
            if (a[j - 1] > a[j])  
                swap(int, a[j - 1], a[j]);  
    }  
}
```

```
int main(void)
```

```
{
```

```
    int nx;
```

```
    puts("버블 정렬");
```

```
    printf("요소 개수: ");
```

```
    scanf("%d", &nx);
```

```
    int* x = calloc(nx, sizeof(int)); // 요소 개수 nx의  
    int형 배열 x 생성
```

```
    for (int i = 0; i < nx; i++) {
```

```
        printf("x[%d] : ", i);
```

```
        scanf("%d", &x[i]);
```

```
    }
```

```
    bubble(x, nx); // 배열 x를 버블 정렬
```

```
    puts("오름차순으로 정렬했습니다.");
```

```
    for (int i = 0; i < nx; i++)
```

```
        printf("x[%d] = %d\n", i, x[i]);
```

```
    free(x);
```

```
    // 배열 해제
```

```
    return 0;
```

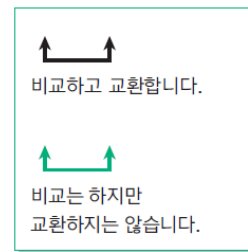
```
}
```



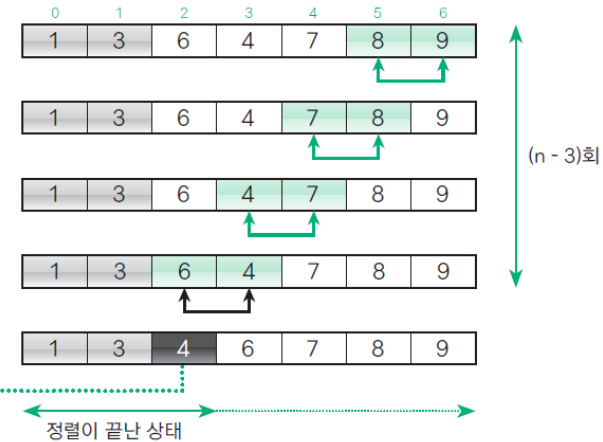
06-2 버블 정렬

◆알고리즘 개선 - (1)

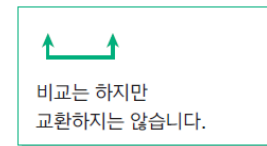
- 이미 배열이 정렬을 마친 상태라면 그 이후의 패스는 요소 교환을 하지 않음
- 즉, 어떤 패스에서 요소의 교환 횟수가 0이면 더 이상 정렬할 요소가 없다는 뜻이기 때문에 정렬 작업을 멈추면 됨



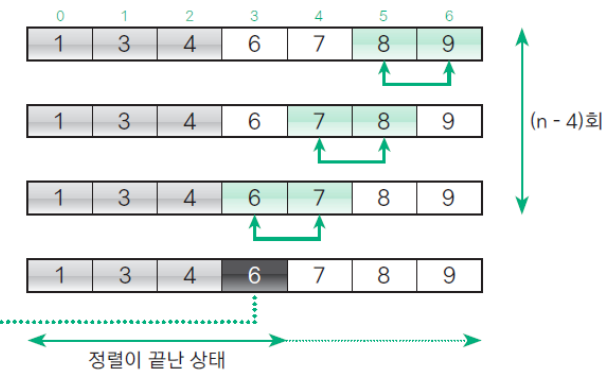
4는 3번째 위치로 이동합니다.



[그림 6-6] 버블 정렬의 세 번째 패스



6은 4번째 위치로 이동합니다.



[그림 6-7] 버블 정렬의 네 번째 패스



06-2 버블 정렬

◆알고리즘 개선 - (2)

- 마지막 교환(★) 이후에 앞쪽의 세 요소({1, 3, 4})는 정렬된 상태
- 어떤 시점 이후에 교환이 수행되지 않는다면 그보다 앞쪽의 요소는 이미 정렬을 마친 상태라고 생각해도 좋음

↑ ↑
비교하고 교환합니다.

↑ ↑
비교는 하지만
교환하지는 않습니다.

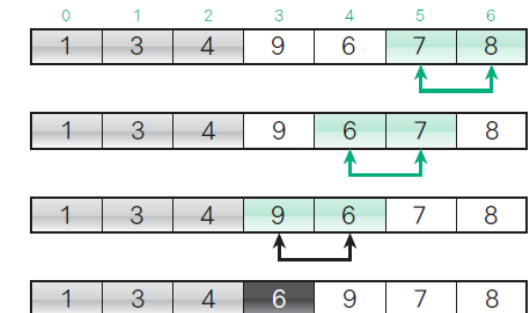
마지막으로 교환을 한 위치 이후의
요소는 정렬을 마친 상태입니다.

↑ ↑
비교하고 교환합니다.

↑ ↑
비교는 하지만
교환하지는 않습니다.



[그림 6-8] 버블 정렬의 첫 번째 패스



[그림 6-9] 버블 정렬의 두 번째 패스



06-2 버블 정렬

◆ 실습 6-3

- 버블 정렬(버전 3)
- 교환을 수행할 때마다 오른쪽 요소의 인덱스 값을 last에 저장하므로, 패스를 마친 시점에 last에는 마지막으로 교환한 두 요소 가운데 오른쪽 요소의 인덱스가 저장됨
- 이 last값을 k에 대입하여 다음에 수행할 패스의 범위를 a[k]까지로 제한
- 다음 패스에서 마지막으로 비교할 두 요소는 a[k]와 a[k + 1]가 됨
- bubble 함수의 시작 부분에서 k 값을 0으로 초기화하는 이유는 첫 번째 패스에서는 모든 요소를 검사해야 하기 때문

Chapt06_실습 6-3/bubble3.c

// 버블 정렬(버전 3: 스캔 범위를 제한)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define swap(type, x, y) do { type t = x; x = y; y = t; } while (0)
```

/*--- 버블 정렬(버전 3: 스캔 범위를 제한) ---*/

```
void bubble(int a[], int n)
```

```
{
```

```
    int k = 0; // a[k]보다 앞쪽의 요소는 정렬을 마친 상태
```

```
    while (k < n - 1) {
```

```
        int last = n - 1; // 마지막으로 교환을 수행한 위치를 저장
```

```
        for (int j = n - 1; j > k; j--)
```

```
            if (a[j - 1] > a[j]) {
```

```
                swap(int, a[j - 1], a[j]);
```

```
                last = j; // 마지막으로 교환을 수행한 위치를 저장
```

```
            }
```

```
            k = last;
```

```
        }
```

```
    }
```



06-3 단순 선택 정렬

◆ 단순 선택 정렬 straight selection sort

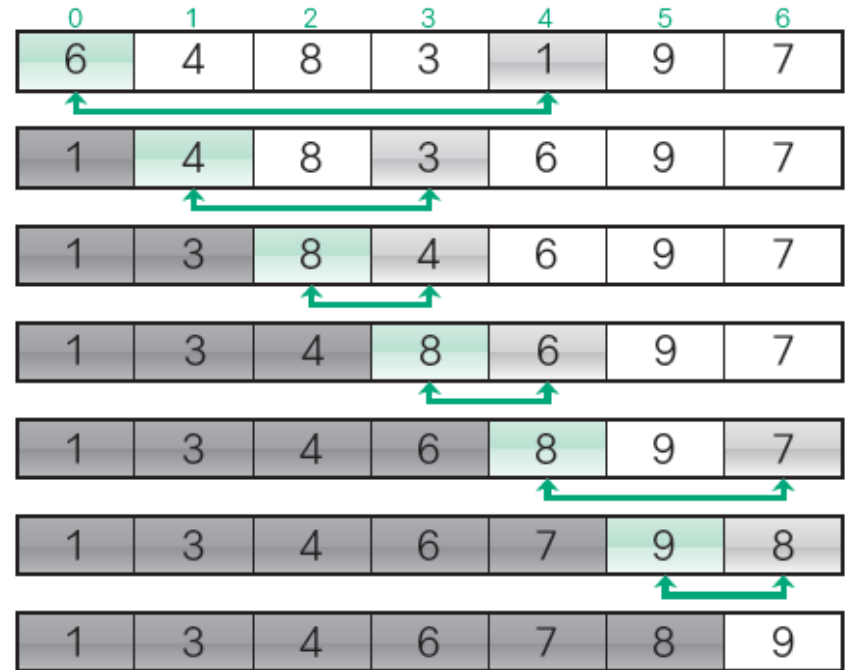
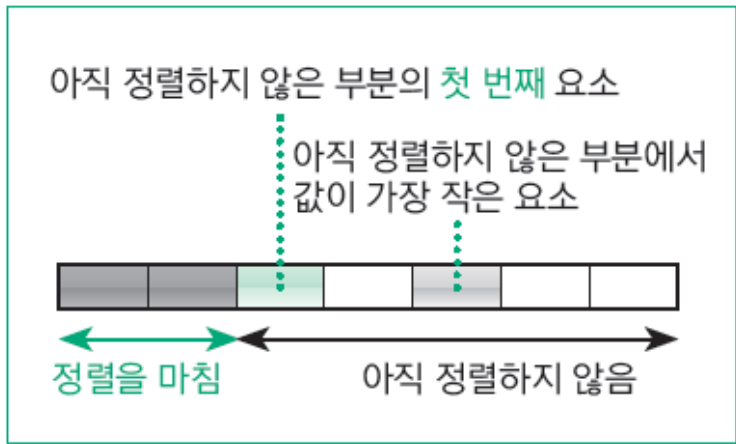
- 아직 정렬하지 않은 부분에서 값이 가장 작은 요소를 선택하고 아직 정렬하지 않은 부분의 첫 번째 요소와 교환

◆ 교환 과정

- 1. 아직 정렬하지 않은 부분에서 가장 작은 키의 값($a[\min]$)을 선택
- 2. $a[\min]$ 과 아직 정렬하지 않은 부분의 첫 번째 요소를 교환



06-3 단순 선택 정렬



[그림 6-10] 단순 선택 정렬 과정



06-3 단순 선택 정렬

◆ 실습 6-4

- 단순 선택 정렬을 수행하는 함수
- 단순 선택 정렬 알고리즘의 요솟값을 비교하는 횟수는 $(n^2 - n) / 2$ 회
- 이 정렬 알고리즘은 서로 떨어져 있는 요소를 교환하는 것이기 때문에 안정적이지 않음

Chapt06_실습 6-4/selection.c

// 단순 선택 정렬

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define swap(type, x, y) do { type t = x; x = y; y = t; } while (0)
```

/*--- 단순 선택 정렬 ---*/

```
void selection(int a[], int n)
```

```
{
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        int min = i;
```

```
        for (int j = i + 1; j < n; j++)
```

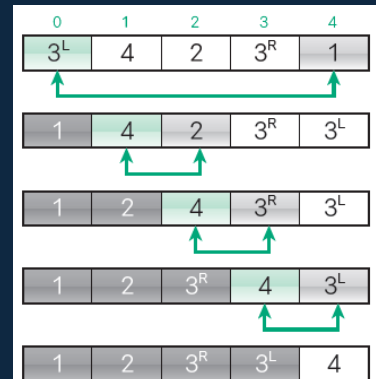
```
            if (a[j] < a[min])
```

```
                min = j;
```

```
        swap(int, a[i], a[min]);
```

```
    }
```

```
}
```



[그림 6-11] 3^L과 3^R의 위치가 뒤바뀌는 과정



06-4 단순 삽입 정렬

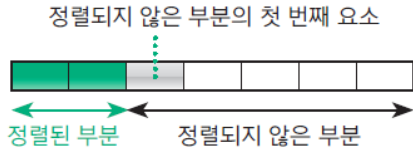
◆ 단순 삽입 정렬 straight insertion sort

- 선택한 요소를 그보다 더 앞쪽의 알맞은 위치에 '삽입하는' 작업을 반복하여 정렬하는 알고리즘
- 아직 정렬되지 않은 부분의 첫 번째 요소를 정렬된 부분의 알맞은 위치에 삽입
- 아래의 두 조건 중 하나를 만족할 때까지 j 를 1씩 감소하면서 대입하는 작업을 반복
 - 1. 정렬된 열의 왼쪽 끝에 도달
 - 2. tmp 보다 작거나 같은 key 를 갖는 항목 $a[j - 1]$ 을 발견
- 드모르간 법칙 적용을 적용해 아래의 두 조건이 모두 성립할 때까지 반복
 - 1. j 가 0보다 큼
 - 2. $a[j - 1]$ 값이 tmp 보다 큼



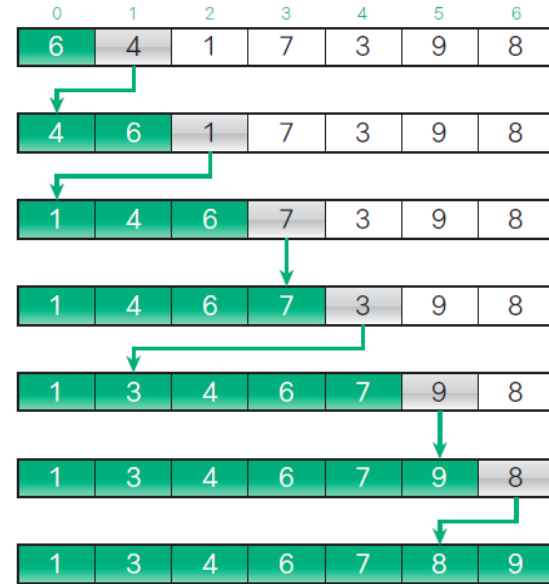
06-4 단순 삽입 정렬

정렬되지 않은 부분의 첫 번째 요소를 정렬된 열의 '알맞은 위치에 삽입'하는 작업을 $n-1$ 회 반복합니다.



정렬된 부분 $a[0], a[1], \dots, a[i-1]$

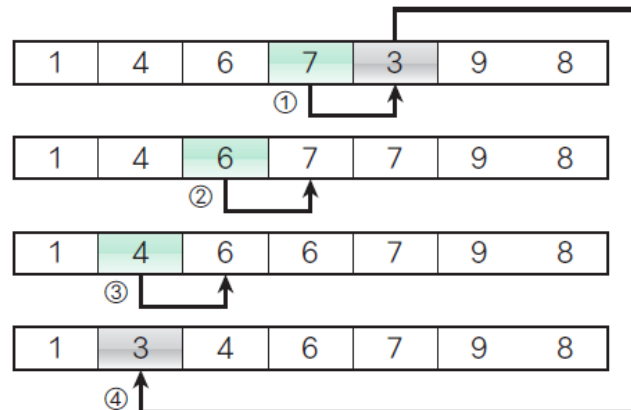
정렬되지 않은 부분 $a[i], a[i+1], \dots, a[n-1]$



[그림 6-12] 단순 삽입 정렬 과정

- ①~③ ... 3보다 작은 요소를 만날 때까지
이웃한 왼쪽의 요소를 대입하는
작업을 반복합니다.
④ ... 멈춘 위치에 3을 대입합니다.

```
j = i;
tmp = a[i];
while (j > 0 && a[j-1] > tmp)
    a[j] = a[j-1];
    j--;
a[j] = tmp;
```



[그림 6-13] 단순 삽입 정렬에서 요소 3의 삽입 과정



06-4 단순 삽입 정렬

◆ 실습 6-5 - (1)

- 단순 삽입 정렬 알고리즘은 떨어져 있는 요소들이 서로 뒤바뀌지 않아 안정적
- 요소의 비교 횟수와 교환 횟수는 $n^2 / 2$ 회

Chapt06_실습 6-5/insertion.c

// 단순 삽입 정렬

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define swap(type, x, y) do { type t = x; x = y; y = t; } while (0)
```

/*--- 단순 삽입 정렬 ---*/

```
void insertion(int a[], int n)
```

```
{
```

```
    for (int i = 1; i < n; i++) {
```

```
        int tmp = a[i];
```

```
        int j;
```

```
        for (j = i; j > 0 && a[j - 1] > tmp; j--)
```

```
            a[j] = a[j - 1];
```

```
        a[j] = tmp;
```

```
    }
```

```
}
```

실행 결과

단순 삽입 정렬

요소 개수: 7

x[0]: 6

x[1]: 4

x[2]: 3

x[3]: 7

x[4]: 1

x[5]: 9

x[6]: 8

오름차순으로 정렬했습니다.

x[0] = 1

x[1] = 3

x[2] = 4

x[3] = 6

x[4] = 7

x[5] = 8

x[6] = 9



06-4 단순 삽입 정렬

◆ 실습 6-5 - (2)

- 단순 삽입 정렬 프로그램

◆ 단순 정렬의 시간 복잡도

- 지금까지 공부한 세 가지 단순 정렬(버블, 선택, 삽입)의 시간 복잡도는 모두 $O(n^2)$ 으로 효율이 좋지 않음

```
int main(void)
{
    int nx;

    puts("단순 삽입 정렬");
    printf("요소 개수: ");
    scanf("%d", &nx);
    int* x = calloc(nx, sizeof(int)); // 요소의 개수가 nx인
    // int형 배열 x를 생성

    for (int i = 0; i < nx; i++) {
        printf("x[%d]: ", i);
        scanf("%d", &x[i]);
    }

    insertion(x, nx); // 배열 x를 단순 삽입 정렬

    puts("오름차순으로 정렬했습니다.");
    for (int i = 0; i < nx; i++)
        printf("x[%d] = %d\n", i, x[i]);

    free(x); // 배열 해제

    return 0;
}
```